

```

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.datasets import make_blobs

# =====

# Step 1: Generate Synthetic Data

# =====

# We use make_blobs to create a dataset with 4 distinct groups (clusters)

# n_samples: Total number of data points

# centers: The true number of clusters in the data

# cluster_std: Standard deviation of the clusters (spread)

# random_state: Ensures reproducibility

X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60, random_state=42)

# Visualize the raw, unclustered data

plt.figure(figsize=(8, 6))

plt.scatter(X[:, 0], X[:, 1], s=50, cmap='viridis')

plt.title("Raw Data (Unclustered)")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()

# =====

# Step 2: Apply K-Means Clustering

# =====

# Initialize the KMeans model with 4 clusters

kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)

```

```

# Fit the model to the data and predict cluster labels
y_kmeans = kmeans.fit_predict(X)

# Get the coordinates of the cluster centers (centroids)
centroids = kmeans.cluster_centers_

# =====

# Step 3: Visualize the Clusters and Centroids
# =====

plt.figure(figsize=(8, 6))

# Plot the data points, colored by their assigned cluster
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis', label='Data Points')

# Plot the centroids as red 'X' marks
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', s=200, marker='X', label='Centroids')

plt.title("K-Means Clustering Results (k=4)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()

# =====

# Step 4: Find the Optimal Number of Clusters (The Elbow Method)
# =====

# In real-world scenarios, we don't know the true number of clusters.
# The Elbow Method helps us find the optimal 'k' by plotting the inertia
# (Within-Cluster Sum of Squares) for different values of k.

```

```
inertia = []

K_range = range(1, 11)

for k in K_range:
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)
    inertia.append(model.inertia_)

# Plot the Elbow Curve
plt.figure(figsize=(8, 6))
plt.plot(K_range, inertia, marker='o', linestyle='--', color='b')
plt.title("The Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia (Within-Cluster Sum of Squares)")
plt.xticks(K_range)
plt.grid(True)
plt.show()
```